Deploying the App on Kubernetes (minikube)

Assignment report: local Kubernetes deployment of the frontend and Flask backend

Objective

Deploy the application from the previous assignment, a frontend and a Flask backend connected to MongoDB Atlas, into a Kubernetes cluster running locally with minikube.

Overview of the approach

The application is split into two workloads, each with its own Kubernetes Deployment and Service, defined in a single manifest, `k8s/todo-app.yaml`:

- `todo-backend`: the Flask API, exposed on port 5000.
- `todo-frontend`: the web frontend, exposed on port 3000.

The MongoDB Atlas connection settings are not baked into the image or the manifest. They are stored in a Kubernetes Secret named `todo-app-secrets`, created from the same local `.env` file used with Docker Compose. The backend image is built directly inside minikube, so no external registry is needed.

Commands used

Step	Command	Purpose
1	`minikube start`	Start the local cluster (Docker driver).
2	`kubectl create secret generic todo-app-secrets --from-env-file=.env --dry-run=client -o yaml | kubectl apply -f -`	Store the Atlas settings from .env as a Secret.
3	`minikube image build -t todo-backend:local -f backend/Dockerfile .`	Build the backend image inside minikube.
4	`kubectl apply -f k8s/todo-app.yaml`	Create both Deployments and Services.
5	`kubectl rollout status deployment/todo-backend`	Wait until the backend is ready.
6	`kubectl port-forward service/todo-backend 5000:5000`	Reach the backend API at localhost:5000.
7	`kubectl rollout status deployment/todo-frontend`	Wait until the frontend is ready.
8	`kubectl port-forward service/todo-frontend 3000:3000`	Reach the frontend at localhost:3000.

Step 1: Start the minikube cluster

minikube v1.39.0 was started with the Docker driver on an existing profile, bringing up a single-node cluster running Kubernetes on containerd. The default storage class and storage provisioner add-ons were enabled, and `kubectl` was configured to use the `minikube` cluster and `default` namespace.

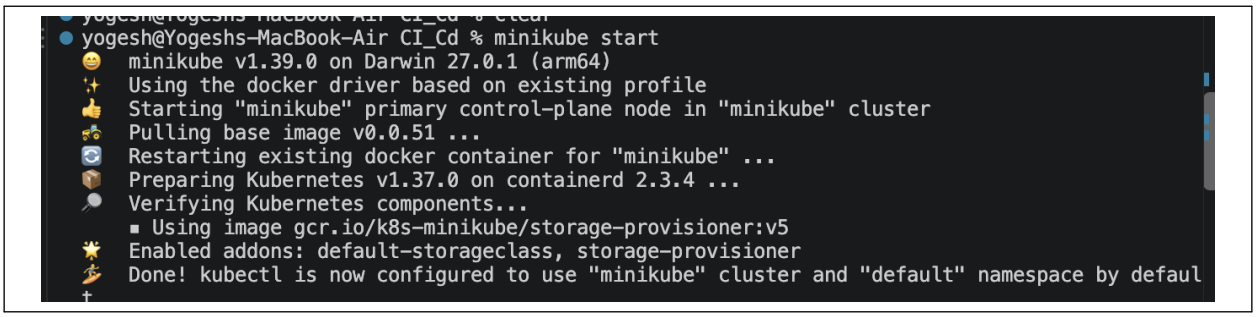


Figure 1: Starting the minikube cluster.

Step 2: Create the Secret and build the backend image

A generic Secret, `todo-app-secrets`, was created from the `.env` file. Using `--dry-run=client -o yaml` piped into `kubectl apply` makes the command safe to re-run when the file changes. The backend image `todo-backend:local` was then built inside minikube from `backend/Dockerfile` (Python 3.12-slim base), so the cluster can use it without pushing to a registry.

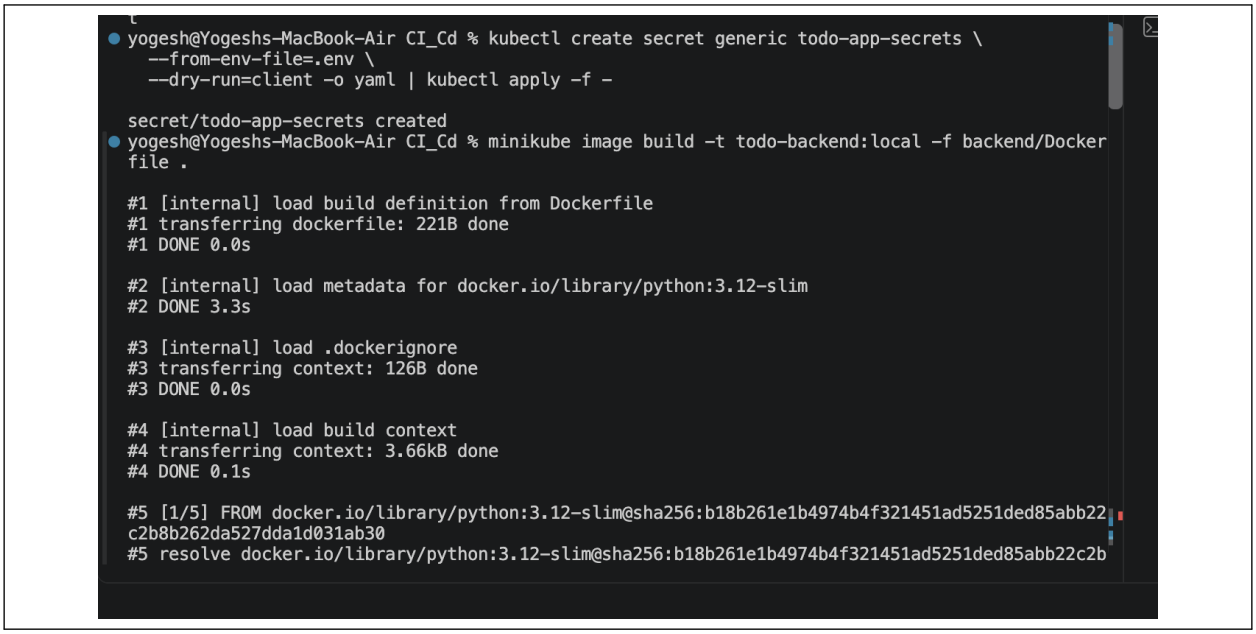


Figure 2: Secret created and backend image build started in minikube.

Step 3: Apply the manifest and deploy the backend

Applying `k8s/todo-app.yaml` created the `todo-backend` and `todo-frontend` Deployments and their Services. The backend rollout completed successfully, and the service was forwarded to `localhost:5000`. The error at the top of the screenshot, `services "todo-backend" not found`, came from an earlier command run before the manifest was applied. Once the Service existed, the problem went away.

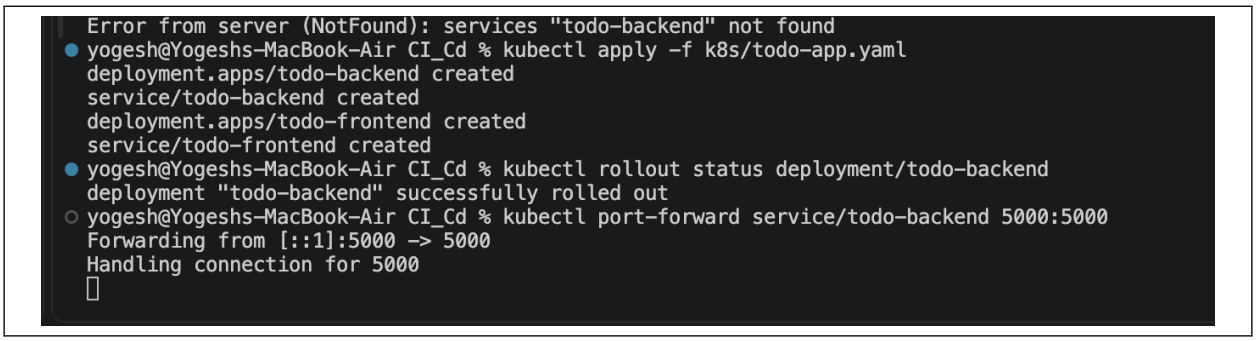


Figure 3: Resources created, backend rolled out and port-forwarded on 5000.

Step 4: Roll out the frontend and expose it

The frontend Deployment initially reported that 0 of 1 replicas were available while its pod started, then finished rolling out. Forwarding `service/todo-frontend` to port 3000 makes the app available in the browser at `http://localhost:3000`; the repeated “Handling connection for 3000” lines show that requests were reaching the service.

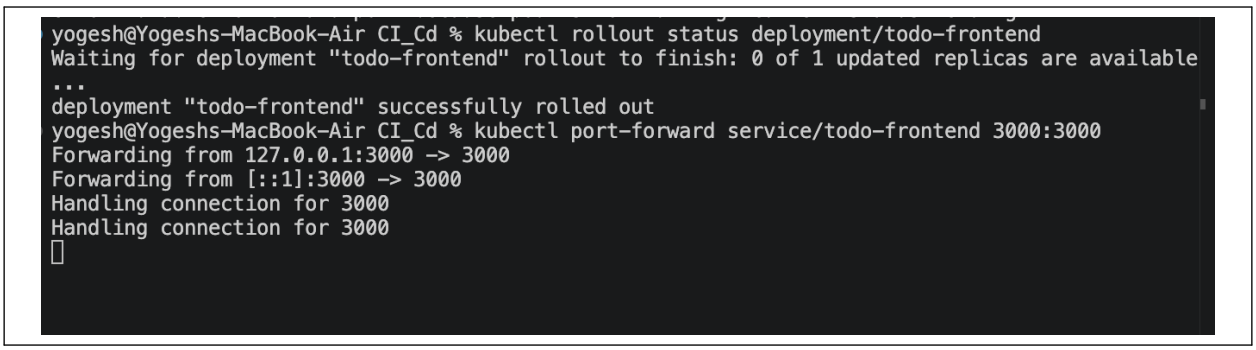


Figure 4: Frontend rollout complete and port-forwarded on 3000.

Conclusion

The application now runs in a local Kubernetes cluster. The frontend and backend each have their own Deployment and Service, the backend image is built inside minikube, and the MongoDB Atlas credentials are supplied through a Kubernetes Secret rather than being hard-coded. Port-forwarding exposes the backend on port 5000 and the frontend on port 3000 for testing from the host machine.